



fides et ratio

PÁZMÁNY

Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar



Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar

MI alapú alkalmazás-fejlesztés

2026 tavasz

Dr. Reguly István Zoltán
reguly.istvan@itk.ppke.hu



Más és nyílt forráskódú LLM-ek



Nyílt forráskódú LLM-ek

- Modellek tanítása, üzemeltetése drága
 - Több billió token tanítás során
 - Több millió dollár betanítani
- Nyílt forráskódú LLM-ek:
 - Megengedő licenz (üzleti felhasználás, amit generálsz, az a tied)
 - Kód és paraméterek: architektúra, tanítási módszer, súlyok elérhetők
 - Testre szabható: finomhangolás
 - Átláthatóság: végig lehet követni hogy jön ki egy válasz (megérteni?)
 - Közösségi innováció: fejlesztől együttműködése, eszközök kidolgozása, szoftveres ökoszisztéma
- Meta LLaMA de-facto standard
 - Mistral AI, Google Gemma



Miért/Mikor használjunk nyílt forráskódú LLM-eket?

- Költség: ismétlődő költségek (tokenenkénti költség) elkerülése – csak a hardverért/áramért kell fizetni
- Testreszabás: céges adatok felhasználása a finomhangolásra
- Átláthatóság: a modell mellett azt is, hogy mi történik az adatoddal
- Jövőbiztos: nem függ külső szolgáltatótól, aki árt, hozzáférést, stb. változtathat egyoldalúan
- Közösségi innováció
- Miért ne?
 - Sosem lesz olyan erős mint a zártak (általános esetben)



Finomhangolás

- Adott egy alap modell, hogy finomhangoljuk?
 - Chat/instruct/specializált modellekké
- Adat
 - Csak szöveg (kiegészítés javítása)
 - Kérdés-válasz párok
- Katasztrofális felejtés
 - Túlzott finomhangolás során elkezd elfelejtteni az alap dolgokat
- Low-Rank Adaptation (LoRA)
 - Alap modell paramétereinek rögzítése
 - Kis mennyiségű plusz paraméter hangolása (adapters)
 - 10000x kevesebb súly



Modell formátumok

- Alapértelmezésben "float" adatformátumban tárolják (fp32, 32-bit)
 - 7B modell: $7e9 * 4 = 28$ GB
- "half" pontosság, 16-bit floating point
- Kvantálás
 - Tovább csonkolják a súlyokat, még kevesebb biten tárolják
 - Tanítás előtt vagy után?
 - QLoRA
- Előny: kisebb méret, skálázhatóbb, gyorsabb "inference"
- Hátrány: pontosságvesztés



Modell formátumok

- PyTorch python könyvtár – legtöbbit használt
 - Súlyok kimentése
 - Tömörítetlen
 - Pickle vagy safetensors fájlok
- Kvantált formátumok
 - GPTQ – 4-bit kvantálás
 - GGUF – dinamikusabb (több-kevesebb bit). CPU, GPU közt szétosztás
 - AWQ – más súlyok más mérettel tárolva
- huggingface.co



Nyílt forráskódú LLM-ek használata

- Tesztelés: <https://arena.ai/>
- Hogyan futtassuk, használjuk ezeket?
 - LM Studio
 - Ollama
 - VLLM
 - Nagyon sok memóriát meg tud enni!
- Üzemeltetést valami informatikusra kell bízni...



Teljesítmény javítása

- Kérdezzünk meg egy közepesen nehéz kérdést egy közepes modelltől 10-szer
 - Többször mond jó választ mint rosszat?
- Hogyan érdemes kérdezni? Visszakérdezni?
 - Zero-shot – amit most csinálunk
 - Few-shot – pár példával

This is awesome! // Negative

This is bad! // Positive

Wow that movie was rad! // Positive

What a horrible show! //



Prompt Chaining

- 2 vagy több lépcsős complex válaszadási feladat
- Példa 1. 1. lépés:
 - Itt egy dokumentum `<dokumentum></dokumentum>` tagek közt:
`<dokumentum>`
`{{DOKUMENTUM}}`
`</dokumentum>`
Kérem, szóról szóra szedd ki a(z) `{{KÉRDÉS}}` kérdéssel kapcsolatos vonatkozó idézeteket. Kérjük, mellékelje az idézetek teljes listáját `<quotes></quotes>` címkékbe. Ha ebben a dokumentumban nincsenek olyan idézetek, amelyek relevánsnak tűnnek ehhez a kérdéshez, kérjük, mondja azt, hogy "Nem találok releváns idézetet".



Prompt Chaining

- 2 vagy több lépcsős complex válaszadási feladat
- Példa 1. 2. lépés:
 - Azt akarom, hogy egy dokumentumot és a dokumentumból származó releváns idézeteket használj fel a kérdés megválaszolásához.
Itt a dokumentum:
<dokumentum>
{{DOKUMENTUM}}
</document>
Íme a kérdés szempontjából leginkább releváns közvetlen idézetek a dokumentumból:
<idézetek>
{{IDÉZETEK}}
</quotes>

Kérem, használja ezeket a „{{KÉRDÉS}}” kérdés megválaszolásához

Győződjön meg arról, hogy válasza pontos, és nem tartalmaz olyan információt, amelyet az idézetek közvetlenül nem támasztanak alá.



Prompt Chaining

- 2. példa 1. rész

- Itt van egy cikk:

- <cikk>

- {{CIKK}}

- </article>

Kérjük, azonosítsa a cikkben található nyelvtani hibákat. Kérjük, csak a hibalistával válaszoljon, semmi mást. Ha nincsenek nyelvtani hibák, mondja azt, hogy „Nincsenek hibák”.



Prompt Chaining

- 2. példa 2. rész

- Itt van egy cikk:

<cikk>

{{CIKK}}

</article>

Kérjük, azonosítsa a cikkben található nyelvtani hibákat, amelyek hiányoznak a következő listából:

<lista>

{{HIBA}}

</list>

Ha a cikkben nincs olyan hiba, amely hiányzik a listából, mondja azt, hogy „Nincsenek további hibák”.



Dokumentumok feldolgozása

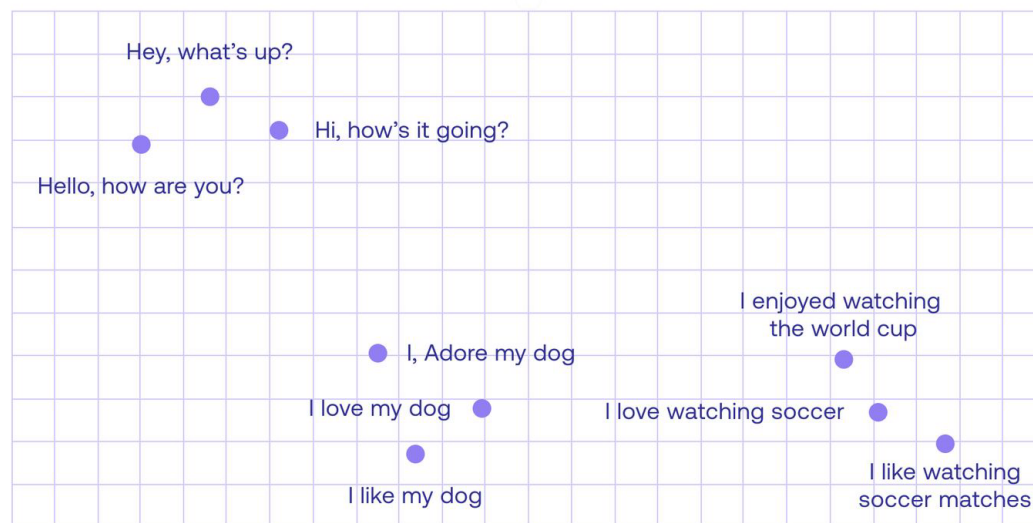
- Ezen technológiák kapcsán felmerül az ötlet, hogy milyen hasznos lenne hosszabb dokumentumokról “beszélgetni”
- Limitáció: context size
 - Hány szót tud feldolgozni az LLM egy kérdésben?
 - GPT 3.5 – 16k token -> 12k szó, 20-30 oldal
 - GPT 4 – 32k/128k token
 - GPT 5 – 400k
 - Claude 2 – 200k token
 - Claude 4.6 – 1M token
 - Google Gemini – 1M token
 - Ár?
 - GPT 4: 10\$ / 1M token – beszélgetés során ez újra és újra átfut rajta
 - Nagyon nagy/sok dokumentum nem fér be, drága is, lassú is



Retrieval Augmented Generation

- Fogjunk egy vagy több dokumentumot, daraboljuk fel (chunk)
- Egy beérkező kérdés kapcsán vegyük elő a releváns darabokat, és azokat adjuk csak át az LLM-nek
- Mi az hogy releváns darab?
 - Hogy döntöm el? Mi jobban, mi kevésbé?
- "Embedding"
 - Mondatok, szövegrészek számokkal való reprezentációja
 - Kérdés számokkal
 - Közel eső szövegrészek

Output





Retrieval Augmented Generation

- Milyen dolgokra lehet ez jó?
 - Keresés (kérdéshez közel eső találatok)
 - Klaszterezés (szövegrészek egymáshoz való hasonlósága)
 - Ajánlás (aktív szöveghez közel eső más részletek – netflix)
 - Anomália detekció (nagyon távol eső részletek)
 - Diverzitás elemzés (hasonlóságok statisztikai tulajdonságai)
 - Osztályozás (szövegrészek a legközelebb eső címkéhez rendelése)



Retrieval Augmented Generation

- Feladat

- 3 fős csapatok
- Válasszunk egy tématerületet, amihez legalább egyikünk ért, és egy ahhoz tartozó hosszabb alapidokumentumot (ha lehet késői 2025-öst, 2026-ost)
 - Kapcsoljuk ki vagy mondjuk meg neki, hogy ne használjon internetes keresést
 - Kérdezzünk bele a dokumentumba, anélkül, hogy feltöltöttük volna (tényleg nem tudja)
- ChatGPT Pro: használjuk a Custom GPT (Explore GPTs/Create) funkciót
- Platform: Assistant Retrieval és feltöltött fájl
- Különösen figyeljünk az instrukciók megadására
- Kérdezzünk bele a dokumentumokba
 - 1-2 könnyebb kérdés - csak egy darab szöveg alapján megválaszolható
 - 3-4 egyre nehezebb kérdés
 - Több szempontból értékeljük – teljesség, logika/érvelés, hivatkozások



Vizuális megértés, válaszadás

- Képek generálása mellett képek megértésére is képesek ezek a modellek
 - ChatGPT: generálj egy képet, amin egy fiú biciklizik egy hosszú zsinóron lufival a kezében egy alacsony aluljáró felé



- Új chat: mi fog várhatóan történni a lufival?
- Képek szerkesztése szövegesen – generáltassunk egy képet, kérjük meg módosítsa - <https://aistudio.google.com/>



Vizuális megértés

- Táblázatok kiemelése PDF-ből
- EMA klinikai teszt: https://www.ema.europa.eu/en/documents/product-information/reagila-epar-product-information_en.pdf
- Table 1 – több oldalra tördelve, cellákon belül is sortörés
- Excel-be kellene átmenteni az adatokat.
 - Excel fájlt nem tud generálni, csak úgy kiírni, hogy utána könnyen bemásolható legyen excelbe
- Próbáljuk ki: chatgpt.com
- <https://aistudio.google.com/>
 - Több különböző modellel is – nekem a Gemini 3.1 Pro teljesített a legjobban
- Megbízhatóság??
 - Szavazás



Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar

AI Ágensek



AI Ágensek

- Egy LLM önmagában semmi mást nem tud, csak szöveget generálni
 - De a szöveg mondhatja azt, hogy oltsam le a lámpát. Vagy küldjek el X-nek, Y tárgyal egy Z szövegű emailt
- Eszközhasználat
 - Elmagyarázzuk az LLM-nek, hogy bizonyos eszközök rendelkezésre állnak, mit lehet velük csinálni, hogy kell használni, mi a várható eredménye
 - “Észrevesszük” ha azt mondja hogy akkor most használjunk egy eszközt
 - Speciális szövegrészlet a válaszában
 - ChatGPT - webes keresés: ha az LLM úgy gondolja, hogy a válaszhoz szükség lenne egy internetes keresésre, akkor keres, az eredményeket elolvassa, és utána ad választ
 - Tetszőleges számítógépes programkód beköthető

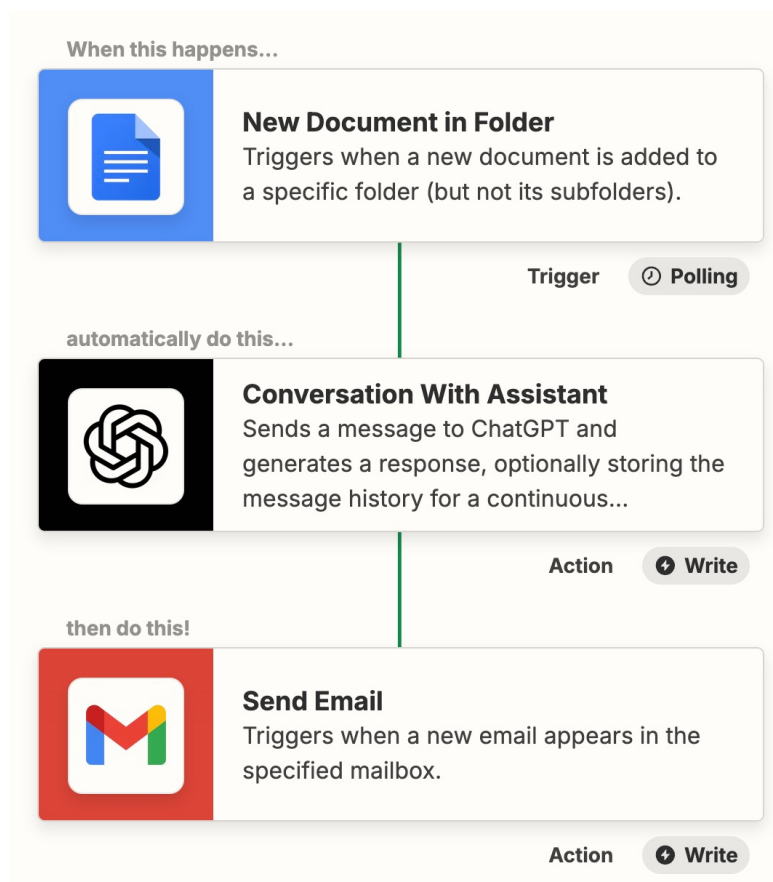


AI Ágensek

- Munkafolyamatok, ahova minimális megértés szükséges
- A kihívás az “integráció” – hogy lehet összekötni levelezéssel, dokumentumokkal, Slack-el, stb
- Sok “no-code” platform
 - <https://n8n.io>
 - <https://zapier.com/>
 - <https://dify.ai/>
 - stb. stb.



AI Ágensek



- [Zapier.com](https://zapier.com)
- Számos webes platformmal integrál
 - Google (email, docs, calendar)
 - Microsoft (Office)
 - Slack
 - Salesforce



Feladat – Zapier – email automatizáció

- Készítsünk egy zapier.com-os “Zap”-et:
 - Elolvassa az új emaileket
 - Eldönti, hogy van-e benne eseményről szó, ha igen, mikor (mettől-meddig) és hol
 - Ha volt eseményről szó (filter)
 - Akkor hozzon létre a google calendarban egy megfelelő eseményt
- További ötletek?
- Jó az nekünk ha hozzáférnek az emailjeinkhez, olvasgatja az AI?
 - Lehet ezt “magunk hostolni” is – pl n8n (és rengeteg más)



Claude Sonnet – Model Context Protocol

- LLM által használható eszközök “standardizált” leírása
 - A “nagy LLM-ek” közül egyelőre Claude Sonnet modell ismeri
 - Desktop alkalmazásukba integrálható
 - Sok más fejlesztést támogató eszköz is tudja használni
- <https://mcp.so/> Lista
- Beszélgetés közben eszközök használata
 - Lokális fájlok elérése
 - Böngésző használat
 - Webes keresés



Vibe coding

- Kódolás, kód írása nélkül
- Egész webes alkalmazások fejlesztése
 - Funkcionális leírást adunk, iteráljuk, legenerálja
 - Érdekes másik LLM-el részletezni a specifikációt, kibontani az alapötletet
 - Más weboldalak stílusát “lopjuk”
- Kell hozzá azért egy alapvető megértés a webes alkalmazások szerkezetéről
 - De persze ezt el tudja magyarázni
- 21st.dev – weblapok elemeinek tárháza
- lovable.dev – teljes webes alkalmazás
- rork.app – mobilos alkalmazások
- copycoder.ai – képek, rajzok alapján alkalmazás implementáció



Feladat

- Alkossunk 3 fős csoportokat
- Promptolva fejlesszünk egy webes-mobilos appot (lovable/rork)
 - Először járjuk körül az ötletet, írjuk le pontosabban mit szeretnénk
 - Minél teljesebb ötlettel menjünk az app fejlesztő rendszerhez
- Pár alkalmazás ötlet
 - Online ötletmegosztó platform. Ötletek kategorizálhatók, egymás ötletére visszajelzés adható, fel/le szavazni lehet őket, iterálni az ötletet
 - Receptmegosztó oldal, ahol felhasználók megoszthatják a kedvenc receptjeiket, képekkel. Egymást lehet követi, like-olni.
 - Film ajánló oldal – mindenki vezetheti magának miket nézett, mi mennyire tetszett. Barátaink kedvenc filmjeit láthatjuk. Filmekhez netes, mozis elérés, műsor automatikusan betölthető



Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar

OpenAI API



OpenAI API

- Mi van a ChatGPT webes interfésze mögött?
- Dolgozhatunk az LLM-ekkel programkódból is
 - Nincs webes interfész
 - Web-alapú lekérésekkel kommunikálunk
 - Szkriptnyelvekből hívjuk az LLM-eket
- Határtalan lehetőségek
- De az elköltött pénz is!
 - GPT-4 0.01\$/1K token bemenet, 0.03\$/1K token kimenet



Beállítás

- “API kulcs” szükséges
 - Minden lekéréssel együtt elküldi a számítógép, ezzel azonosít a GPT
 - Biztos helyre kell tenni – ne legyen publikusan elérhető
 - Nem lehet újra megnézni
 - Visszavonható
- <https://platform.openai.com/account/api-keys>
- Ma nem kell



Hello world

- Egyszerű webes lekérés cURL segítségével
 - Parancssoros eszköz HTTP/HTTPS lekérések indítására

```
curl https://api.openai.com/v1/chat/completions -H "Content-Type: application/json" -H  
"Authorization: Bearer $OPENAI_API_KEY" -d '{  
  "model": "gpt-4o-mini",  
  "messages": [  
    {  
      "role": "system",  
      "content": "Egy versíró művész vagy, aki hétköznapi helyzetek alapján ír színes,  
elgondolkodtató verseket."  
    },  
    {  
      "role": "user",  
      "content": "Írj egy verset a hétvégi korán kelés fájdalmáról."  
    }  
  ]  
'
```




Munkakörnyezet

- Új notebook indítása
- Forráskódokat innen másolhatunk:
- <https://github.com/reguly/aiappdev>
- <https://github.com/reguly/aiappdev/blob/main/Programming.ipynb>



OpenAI python környezetből

- !pip install openai
- -OPENAI_API_KEY környezeti változó (Teams-ből)

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Egy versíró művész vagy, aki hétköznapi helyzetek alapján ír színes, elgondolkodtató verseket."},
        {"role": "user", "content": "Írj egy verset a hétfői korán kelés fájdalmáról."}
    ]
)

print(completion.choices[0].message.content)
```



Szöveg generáló modellek

- Szöveges (vagy egyéb, pl kép) bemenet, szöveges kimenet
- Mire lehet használni:
 - Píszkozat dokumentumok készítése
 - Kód írása
 - Kérdések megválaszolása egy tudásbázisból
 - Szövegek elemzése
 - Összetett szoftverekhez szöveges interfész biztosítása
 - Különböző alap témák magyarázata
 - Fordítása
 - Karaterek, ágensek különböző helyzetekben – customer service
- gpt-5: képeket is fel tud dolgozni, meg tud érteni



Chat Completions

- Chat jellegű működése:

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-5-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Who won the world series in 2020?"},
        {"role": "assistant", "content": "The Los Angeles Dodgers won the World Series in 2020."},
        {"role": "user", "content": "Where was it played?"}
    ]
)
response.model_dump()
```

- Mivel az API (<https://api.openai.com/v1/chat/completions>) nem emlékszik az előző üzenetekre
- Az előzményeket is el kell küldeni (költség!)
- Lásd még a Threads API-t



Válasz

- Mi jön vissza a válaszban?

```
{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "message": {
        "content": "The 2020 World Series was played in Texas at Globe Life Field in Arlington.",
        "role": "assistant"
      },
      "logprobs": null
    }
  ],
  "created": 1677664795,
  "id": "chatcmpl-7QyqpwhfhqwjicIEznoc6Q47XAYW",
  "model": "gpt-3.5-turbo-0613",
  "object": "chat.completion",
  "usage": {
    "completion_tokens": 17,
    "prompt_tokens": 57,
    "total_tokens": 74
  }
}
```

stop/length/function_call/content_filter/null

cost! \$\$\$

print(completion.choices[0].message.content)



Programozási gyakorlat

- **AI Chatbot Fejlesztése**

- **Cél:** Kérdés-válasz dialógust implementáló chatbot készítése az OpenAI API segítségével, ami egy konkrét témakörben felmerülő kérdésekre válaszol, a témakörön kívüli kérdéseket elutasítja. Pl. HR tanácsadó, aki álláshirdetéseket segít írni, és jelentkezőket értékelni.
- **Feladatok:**
 - Definiáljuk a chatbot célját és területét.
 - Implementáljuk a beszélgetés folyamatát az OpenAI API segítségével.
 - Teszteljük a chatbot viselkedését, teljesítményét.



JSON mód

- Van hogy struktúrált kimenetre van szükségünk, hogy utána kódból könnyen tudjuk kezelni

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-5-mini",
    response_format={ "type": "json_object" },
    messages=[
        {"role": "system", "content": "You are a helpful assistant designed to output JSON."},
        {"role": "user", "content": "Who won the world series in 2020?"}
    ]
)
print(response.choices[0].message.content)

import json
my_dict = json.loads(response.choices[0].message.content)
print(my_dict)
print(my_dict['winner'])
print(my_dict['year'])
```



Függvényhívás

- Motiváció: információ szabad szövegben, ennek strukturált adattá való alakítása. Pl CV-ből “kezelhető adat”

```
student_1_description = "David Nguyen is a sophomore majoring in computer science at Stanford University. He is Asian American and has a 3.8 GPA. David is known for his programming skills and is an active member of the university's Robotics Club. He hopes to pursue a career in artificial intelligence after graduating."
```

```
# A simple prompt to extract information from "student_description" in a JSON format.
```

```
prompt_1 = f'''
```

```
Please extract the following information from the given text and return it as a JSON object:
```

```
name
major
school
grades
club
```

```
This is the body of text to extract the information from:
```

```
{student_1_description}
'''
```

```
# Generating response back from gpt-3.5-turbo
```

```
openai_response = client.chat.completions.create(
    model = 'gpt-5-mini',
    messages = [{'role': 'user', 'content': prompt_1}]
)
```

```
print(openai_response.choices[0].message.content)
```




Függvényhívás

- LLM interakciója a “világgal”
 - Nyiss meg egy fájlt, kérdd le az időjárást, kapcsold be a kamerát, stb.
 - LLM-eket utasítunk függvények hívására
 - Strukturált módon le kell írni mi a függvény, mit csinál, milyen paraméterei vannak
 - A függvényt már nekünk kell meghívni
1. LLM meghívása a felhasználó kérdésével, valamint a hívható függvények listájának átadásával
 2. Az LLM úgy dönthet, hogy egy vagy több függvényt meghív. Ebben az esetben egy JSON kimentet ad, amiben a függvény és paramétereit megmondja. Esetenként hallucinálhat paramétereket.
 3. JSON értelmezése a saját kódunkban, függvény meghívása a megadott paraméterekkel
 4. LLM újbóli meghívása a függvény kimenetével – az LLM ezek után összegzi az eredményeket



Függvényhívás

```
def submit_student_info(name, major, school, grades, club):  
    print(f"Student Name: {name}")  
    print(f"Major: {major}")  
    print(f"School: {school}")  
    print(f"Grades: {grades}")  
    print(f"Club: {club}")  
  
    student_custom_functions = [  
        {  
            'name': 'submit_student_info',  
            'description': 'Submit student information for processing',  
            'parameters': {  
                'type': 'object',  
                'properties': {  
                    'name': {  
                        'type': 'string',  
                        'description': 'Name of the person'  
                    },  
                    'major': {  
                        'type': 'string',  
                        'description': 'Major subject.'  
                    },  
                    'school': {  
                        'type': 'string',  
                        'description': 'The university name.'  
                    },  
                    'grades': {  
                        'type': 'integer',  
                        'description': 'GPA of the student.'  
                    },  
                    'club': {  
                        'type': 'string',  
                        'description': 'School club for extracurricular activities.'  
                    }  
                }  
            }  
        }  
    ]
```



Függvényhívás

```
prompt1 = '''
I need to submit structured student information for processing to a database. This is the body of
text to extract the information from:
'''

student_description = [prompt1+student_1_description,prompt1+student_2_description]
for i in student_description:
    response = client.chat.completions.create(
        model = 'gpt-5-mini',
        messages = [{'role': 'user', 'content': i}],
        functions = student_custom_functions,
        function_call = 'auto'
    )

    # Loading the response as a JSON object
    print(response.model_dump())
    json_response = json.loads(response.choices[0].message.function_call.arguments)
    print(json_response)
    submit_student_info(**json_response)
```



Feladat – eszközhasználó AI ágens

- **Névjegykártyák feldolgozása**
- Mik a mezők?



Feladat – eszközhasználó AI ágens

- **Saját ötlet?**
- **Befektetési tanácsadó**
 - **Cél:** Egy AI-alapú ágens létrehozása mely a közelmúlt tőzsdei adatai alapján különböző cégekbe való befektetésről képes tanácsot adni.
 - **Lépések:**
 - Hozzunk létre egy új "chatbot"-ot, ami képes a beszélgetést tárolni, újra elküldeni
 - Adjunk neki releváns "system" leírást
 - Hozzunk létre egy yfinance python csomagot használó függvényt és a hozzá tartozó leírást, ami egy adott tőzsdei szimbólumhoz (pl AAPL) le tudja kérni az elmúlt 30 nap árait
 - Konfiguráljuk a chatbotot ezzel a "tool"-al
- Próbáljuk ki, kérdezzük meg milyen trendeket lát, érdemes-e befektetni
- Mik a tapasztalatok?



OpenAI API

- Sok mást tud még
- Finomhangolás
- Képgenerálás (DALL-E)
- Képek elemzése
- Szöveg beszéddé alakítása és vissza
- Assistants
 - Threads
 - Eszközök: Code interpreter, Knowledge Retrieval, Function Calling



Projektfeladat 2

- Ágenses dokumentum megértés, formanyomtatvány kitöltés.
- Cél: tetszőleges docx Word dokumentumban üres mezők megtalálása, megértése, majd adott mappában lévő dokumentumokból ezen információk kinyerése, és a form kitöltése, az eredeti formázás megtartásával
- Form: [példa1](#), [példa2](#) – működjön legalább 2 félére
- Adatok: akár saját, akár tetszőlegesen generáltak. Több fájlból, legyen közte kép is.
 - Generálásra példa: “Tanító adatot kell generálnom, személyi igazolványon adatok felismeréséhez. Generálj nekem egy minta magyar személyi igazolványt”
- Legyen teljesen automatikus a folyamat, bemeneti űrlapot és mappát (az adatokkal) kelljen megadni
- Beadandó: python kód vagy colab notebook.